

MINI PROJECT

SINSHA C
21/05/2026

AWS Multi-region VPC Peering and Network Setup.

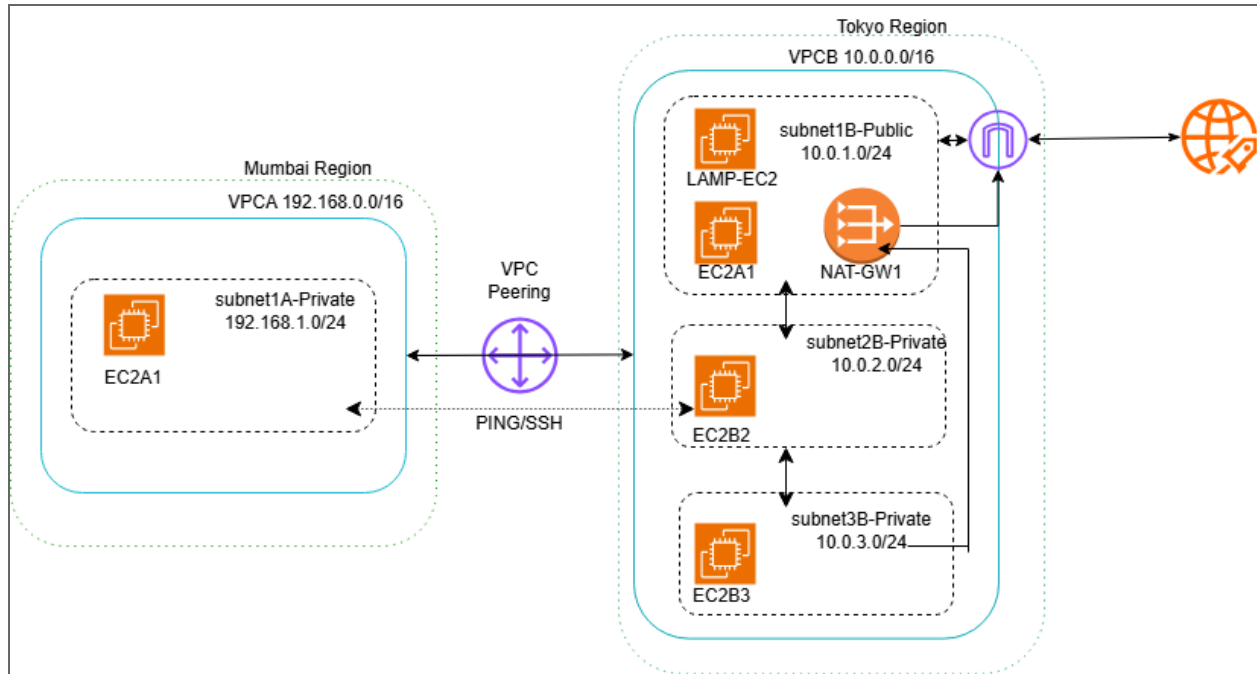
Overview

This project demonstrates how to build a secure multi-region architecture using Amazon VPC, subnets, EC2 instances, and VPC Peering across two AWS regions: Mumbai and Tokyo.

We will design:

- VPC A (Mumbai) → Private workload
- VPC B (Tokyo) → Public + Private segmented architecture
- Cross-region VPC Peering
- Controlled internet access using NAT Gateway
- EC2 instances in each subnet (including LAMP stack server)

Architecture diagram



Step 1: Create VPCs

- ☐ Create VPC A (Mumbai)
 - ☐ Login to AWS and select the service VPC.
 - ☐ Create a VPC named VPCA with CIDR 192.168.0.0/16
 - ☐ Create subnet1A-Private with CIDR 192.168.1.1/24
 - ☐ Create Route table RTA and associate with subnet1A-Private

VPC > Your VPCs > Create VPC

A VPC is an isolated portion of the AWS Cloud populated by AWS objects, such as Amazon EC2 instances.

VPC settings

Resources to create [Info](#)

Create only the VPC resource or the VPC and other networking resources.

☒ VPC only

☐ VPC and more

Name tag - *optional*

Creates a tag with a key of 'Name' and a value that you specify.

VPCA

IPv4 CIDR block [Info](#)

☒ IPv4 CIDR manual input

☐ IPAM-allocated IPv4 CIDR block

IPv4 CIDR

192.168.0.0/16

CIDR block size must be between /16 and /28.

VPC > Subnets > Create subnet

Subnet name

Create a tag with a key of 'Name' and a value that you specify.

subnet1A-Private

The name can be up to 256 characters long.

Availability Zone [Info](#)

Choose the zone in which your subnet will reside, or let Amazon choose one for you.

Asia Pacific (Mumbai) / ap-s1-az1 (ap-south-1a)

IPv4 VPC CIDR block [Info](#)

Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

192.168.0.0/16

IPv4 subnet CIDR block

192.168.1.0/24

256 IPs

< > ^ v

VPC > Route tables > rtb-00d3009ae4c8bfad1 > Edit subnet associations

Edit subnet associations

Change which subnets are associated with this route table.

Available subnets (1/1)

Filter subnet associations

	Name	Subnet ID	IPv4 CIDR	IPv6 CIDR	Route table ID
☒	subnet1A-Private	subnet-0a5413c20187af12a	192.168.1.0/24	–	Main (rtb-06896d09e6cebc9e5)

Selected subnets

subnet-0a5413c20187af12a / subnet1A-Private X

Cancel Save associations

☐ Create VPC B (Tokyo)

- ☐ Change the region to Tokyo and create another VPC named VPCB with CIDR 10.0.0.0/16
- ☐ Create subnets
 - ☐ subnet1B-Public with CIDR 10.0.1.0/24
 - ☐ subnet2B-Private with CIDR 10.0.2.0/24
 - ☐ subnet3B-Private with CIDR 10.0.3.0/24
- ☐ Create corresponding route table and associate with subnets RTB1,RTB2 and RTB3
- ☐ Create Internet gateway IGW1

VPC > Internet gateways > Create internet gateway

Create internet gateway Info

An internet gateway is a virtual router that connects a VPC to the internet. To create a new internet gateway specify the name for the gateway below.

Internet gateway settings

Name tag
Creates a tag with a key of 'Name' and a value that you specify.

IGW1

Tags - optional
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key	Value - optional	
Q Name X	Q IGW1 X	Remove

Add new tag

You can add 49 more tags.

Cancel Create internet gateway

☐ Attach IGW1 to VPCB

VPC > Internet gateways > Attach to VPC (igw-00ae6c541361377ad)

Attach to VPC (igw-00ae6c541361377ad) Info

VPC
Attach an internet gateway to a VPC to enable the VPC to communicate with the internet. Specify the VPC to attach below.

Available VPCs
Attach the internet gateway to this VPC.

Q vpc-0df01844b62fd8add

► AWS Command Line Interface command

Cancel Attach internet gateway

☐ Go to the route table associated with subnet1B-Public. Edit route and add igw id as route

VPC > Route tables > rtb-0d270bb890c0eef14 > Edit routes

Edit routes

Destination	Target	Status	Propagated	Route Origin
10.0.0.0/16	local	Active	No	CreateRouteTable
Q 0.0.0.0/0	Internet Gateway	-	No	CreateRoute
	Q igw-00ae6c541361377ad			
	Use: "igw-00ae6c541361377ad"			
	igw-00ae6c541361377ad (IGW1)			

Add route Remove

Cancel Preview Save changes

Step 2: Launch EC2 Instances

Prerequisites

- ☐ Login to both region and create keypair
- ☐ Create security group which allow ssh, ICMP and HTTP

1. In VPCA (Mumbai region):

- Launch 1 EC2 instance name: EC2A
- Place it in private subnet (192.168.1.0/24)

2. In VPCB (Tokyo region):

- Launch 3 EC2 instances:

Subnet	EC2 Name
subnet1B-Public 10.0.1.0/24	EC2B1
subnet2B-Private 10.0.2.0/24	EC2B2
subnet3B-Private 10.0.3.0/24	EC2B3

- NB: here I have used Ubuntu 26 for EC2B1 for Lamp and auto assign public should be enabled.

STEP 3: Create NAT Gateway

- ☐ Create NAT Gateway (name: NAT-GW1) in Subnet1B-Public (10.0.1.0/24) in the Tokyo region.

☰ VPC > NAT gateways > Create NAT gateway

✓ Elastic IP address 57.180.72.71 (eipalloc-01f0a1131917680f7) allocated.

Name - optional
Create a tag with a key of 'Name' and a value that you specify.

NAT-GW1

The name can be up to 256 characters long.

Availability mode [Info](#)
Choose whether to deploy across all zones in the region or restrict to a single availability zone.

☐ Regional - new
Scales automatically across all regional AZs, simplifying management for multi AZ deployments.

☒ Zonal
Provides granular control within a specific availability zone, adhering to subnet level settings.

Subnet
Select a subnet in which to create the NAT gateway.

subnet-03c698fd9efa874b5 (subnet1B-Public)

Connectivity type
Select a connectivity type for the NAT gateway.

☒ Public

☐ Private

Elastic IP allocation ID [Info](#)
Assign an Elastic IP address to the NAT gateway.

eipalloc-01f0a1131917680f7

☐ [Allocate Elastic IP](#)

- ☐ After creating NAT-GW1 edit Route table RTB3(associated with subnet3B-Private) and add NAT gateway

☰ VPC > Route tables > rtb-0423989e09d7e81ce > Edit routes

Edit routes

Destination	Target	Status	Propagated	Route Origin
10.0.0.0/16	local	Active	No	CreateRouteTable
0.0.0.0/0	NAT Gateway	-	No	CreateRoute

[Add route](#) [Cancel](#) [Preview](#) [Save changes](#)

- ☐ Means Outbound internet allowed and Inbound internet blocked

Step 4: VPC Peering (Mumbai ↔ Tokyo)

Create cross-region peering using Amazon VPC

4.1 Create Peering Connection

- Request peering:
 - VPC A (Mumbai)

VPC > Peering connections > Create peering connection

Select another VPC to peer with

Account

☒ My account
☐ Another account

Region

☐ This Region (ap-south-1)
☒ Another Region

Asia Pacific (Tokyo) (ap-northeast-1)

VPC ID (Acceptor)

vpc-0df01844b62fd8add

- After creating peering request Status will be shown as *Pending Acceptance by ACCOUNTID*
- Accept requests from the VPCB, Tokyo region.

VPC > Peering connections

Peering connections (1/1) info

Find peering connections by attribute or tag

Name	Peering connection ID	Status	Requester VPC
-	pcx-04f660df9658c7285	Pending acceptance	vpc-01bc02713b56071d

Actions: View details, Accept request, Reject request, Edit DNS settings, Manage tags, Delete peering connection

- Edit route of the route table of RTB2
- Add destination as Subnet1A-private CIDR

VPC > Route tables > rtb-097d4b7716995c963 > Edit routes

Edit routes

Destination	Target	Status	Propagated	Route Origin
10.0.0.0/16	local	Active	No	CreateRouteTable
192.168.1.0/24	Peering Connection	-	No	CreateRoute

pcx-04f660df9658c7285

Add route

Cancel Preview Save changes

- Target peering connection and save changes.

9

- Go to Mumbai region and edit route table RTA1 add destination as subnet2B-Private

☰ VPC > Route tables > rtb-00d3009ae4c8bfad1 > Edit routes

Edit routes

Destination	Target	Status	Propagated	Route Origin
192.168.0.0/16	local	Active	No	CreateRouteTable
Q 10.0.2.0/24	Peering Connection	Active	No	CreateRoute
Q pcx-04f660df9658c7285				

[Add route](#)
[Cancel](#)
[Preview](#)
[Save changes](#)

- And save changes

Step 5: Test Peering Connection

- From EC2B1 (VPCB) ssh to EC2B2
 - Copy the tokyo keypair private file to EC2B1 as key.pem and ssh
 - Chmod 400 key.pem
 - Ssh -i key.pem ec2-user@<privateip-of-EC2B2 instance>

- Private-to-private communication works via peering

Test NAT Gateway

SSH to EC2B3 from EC2B1 and ping [google.com](https://www.google.com)

```

_/m/'
[ec2-user@ip-10-0-3-129 ~]$ ping google.com
PING google.com (142.250.21.100) 56(84) bytes of data.
64 bytes from zh-in-f100.1e100.net (142.250.21.100): icmp_seq=1 ttl=110 time=3.28 ms
64 bytes from zh-in-f100.1e100.net (142.250.21.100): icmp_seq=2 ttl=110 time=2.68 ms
64 bytes from zh-in-f100.1e100.net (142.250.21.100): icmp_seq=3 ttl=110 time=2.67 ms
64 bytes from zh-in-f100.1e100.net (142.250.21.100): icmp_seq=4 ttl=110 time=2.66 ms
64 bytes from zh-in-f100.1e100.net (142.250.21.100): icmp_seq=5 ttl=110 time=2.65 ms
64 bytes from zh-in-f100.1e100.net (142.250.21.100): icmp_seq=6 ttl=110 time=2.69 ms
64 bytes from zh-in-f100.1e100.net (142.250.21.100): icmp_seq=7 ttl=110 time=2.64 ms
64 bytes from zh-in-f100.1e100.net (142.250.21.100): icmp_seq=8 ttl=110 time=2.66 ms
64 bytes from zh-in-f100.1e100.net (142.250.21.100): icmp_seq=9 ttl=110 time=2.66 ms
64 bytes from zh-in-f100.1e100.net (142.250.21.100): icmp_seq=10 ttl=110 time=2.85 ms
64 bytes from zh-in-f100.1e100.net (142.250.21.100): icmp_seq=11 ttl=110 time=2.69 ms
^C

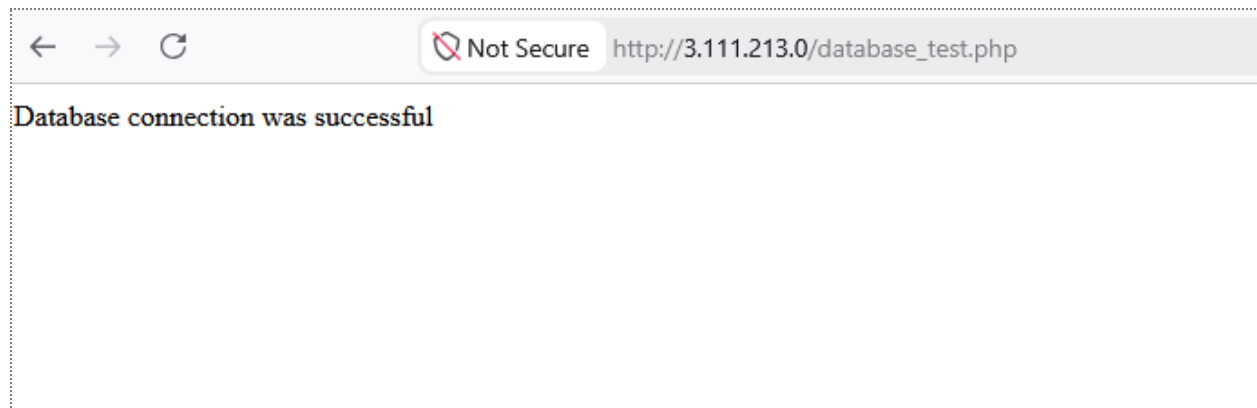
```

It works!

Step 6: LAMP Stack Setup (VPC B Public Subnet)

- Launch an EC2 instance in VPCB Public subnet named LAMP-EC2
- Ssh to LAMP-EC2 and Install LAMP stack
- Follow the link below for LAMP installation.
 - <https://docs.vultr.com/install-linux-apache-mysql-and-php-lamp-on-ubuntu-20-04-lts>
 - Then, visit the address below:

http://<public ip of your instance>/database_test.php
 - You should get the output below that shows that the script has successfully connected to the database.



Conclusion

This project demonstrates a secure and scalable multi-region AWS architecture using VPCs in Mumbai and Tokyo. It includes proper subnet segmentation, EC2 deployment across public and private subnets, VPC peering for cross-region communication, and NAT Gateway for controlled internet access. Overall, it helps understand real-world cloud networking, security, and connectivity design in AWS.